Реакторная физика рабочий словарь

Термины, которые нужны, чтобы считать активную зону и понимать, что именно ты считаешь. Каждый термин — с объяснением простыми словами и с тем, как он выглядит в коде OpenMC.

> Для проекта: интерактивный симулятор активной зоны (OpenMC → суррогатная модель → 3D в браузере)

1. Основы: что вообще происходит

Семь слов, без которых дальше ничего не читается.

Нуклид / изотоп

Атом характеризуется числом протонов (это химический элемент) и числом нейтронов. Изотопы — атомы одного элемента с разным числом нейтронов. Химически они неразличимы, а для нейтронов — это совершенно разные мишени. Число в названии (U-235, U-238) — сумма протонов и нейтронов.

> Главное для тебя: вся реакторная физика работает не с «ураном», а с конкретными изотопами. U-235 и U-238 ведут себя противоположно.

Деление (fission)

Нейтрон попадает в тяжёлое ядро, ядро разваливается на два осколка. Выделяется энергия (это и есть тепло станции) и вылетают 2–3 новых нейтрона. Именно новые нейтроны делают реакцию самоподдерживающейся.

Цепная реакция

Нейтроны от одного деления вызывают следующие деления. Три исхода: цепочка затухает, держится ровно, или нарастает. Вся инженерия реактора — про удержание второго варианта.

U-235 и U-238

U-235 делится от нейтрона любой энергии, особенно охотно от медленного. Это рабочий изотоп. U-238 от обычных реакторных нейтронов не делится — он их поглощает и превращается в плутоний. В природном уране U-235 всего 0,7%, остальное — U-238.

Обогащение

Доля U-235 среди всех атомов урана, в процентах. Природное — 0,7%, для энергетического реактора поднимают до 3–5%. Это **характеристика материала**, а не сам материал: в коде это одно число рядом с плотностью.

В программе (OpenMC):

```
fuel = openmc.Material(name='UO2')
fuel.add_element('U', 1.0, enrichment=4.0)   # 4 % U-235
fuel.add_element('O', 2.0)
fuel.set_density('g/cm3', 10.5)
```

Плутоний-239

Рождается прямо в работающем реакторе: U-238 ловит нейтрон и через пару превращений становится Pu-239, который тоже делится. К концу работы топлива заметная часть энергии идёт уже от него. Поэтому состав топлива меняется со временем сам по себе.

Осколки деления

Обломки развалившегося ядра — десятки новых элементов. Часть из них жадно поглощает нейтроны и мешает цепочке. Самый важный — ксенон-135.

2. Железо: из чего собран реактор

Матрёшка снизу вверх: от таблетки до активной зоны.

Таблетка (pellet)

Само топливо. Цилиндрик спечённой керамики, примерно 1 см в диаметре и 1 см в высоту, чёрно-серый, тяжёлый. Одна такая даёт энергии примерно столько же, сколько тонна угля.

Диоксид урана, UO_2

Материал таблетки. Не металл, а керамика: выдерживает 2800 °C, тогда как в центре таблетки при работе около 1000–1500 °C. Плотность около 10,5 г/см³. Платим за это плохой теплопроводностью — центр намного горячее края.

Твэл (fuel rod / fuel pin)

Столбик таблеток в герметичной трубке длиной около 4 метров. В большом реакторе таких трубок порядка 50 тысяч.

Оболочка (cladding)

Та самая трубка. Делается из циркониевого сплава, потому что цирконий почти не поглощает нейтроны — сталь бы их съедала. Её задача — не выпустить осколки деления наружу.

Кассета / ТВС (fuel assembly)

Пучок из сотен твэлов, собранный в квадратную или шестигранную решётку. Единица, которую физически загружают и выгружают из реактора.

Активная зона (core)

Все кассеты вместе — то место, где идёт цепная реакция. В большом энергоблоке это 150–200 кассет.

Управляющие стержни (control rods)

Стержни не из урана, а из поглотителя — бор, кадмий, гафний. Задвинул глубже — нейтроны съедаются, мощность падает. Выдвинул — растёт. Это единственная ручка регулировки мощности. При потере питания падают вниз сами и глушат реактор.

> Для твоей демки это главный входной параметр: слайдер «глубина стержней» → пересчёт → новая картинка поля нейтронов.

Замедлитель (moderator)

Вещество, которое тормозит быстрые нейтроны до медленных. В большинстве реакторов это обычная вода. Нужен потому, что медленный нейтрон U-235 ловит в сотни раз охотнее. Без замедлителя 4% обогащения не хватит — цепочка не пойдёт.

Теплоноситель (coolant)

То, что уносит тепло из зоны. В водяных реакторах это та же вода, что и замедлитель — одно вещество выполняет обе роли. Отсюда важный эффект: потеряли воду — потеряли и охлаждение, и замедление одновременно.

Отражатель (reflector)

Слой вокруг зоны, возвращающий вылетающие нейтроны обратно. Экономит топливо — без него утечка с краёв была бы заметной.

Выгорающий поглотитель

Поглотитель (обычно гадолиний или бор), подмешанный в топливо специально. В начале он душит избыточную реактивность свежего топлива, а со временем сам выгорает и перестаёт мешать. Способ выровнять поведение реактора в течение кампании.

Вся геометрия в программе (ячейка: таблетка — оболочка — вода):

```
r_fuel = openmc.ZCylinder(r=0.41)      # радиус таблетки, см
r_clad = openmc.ZCylinder(r=0.48)      # внешний радиус оболочки

pellet = openmc.Cell(fill=fuel,  region=-r_fuel)
clad   = openmc.Cell(fill=zirc,  region=+r_fuel & -r_clad)
water  = openmc.Cell(fill=h2o,   region=+r_clad & inside_box)
```

3. Величины: что именно ты считаешь

Числа, которые выдаёт расчёт и которые ты будешь показывать.

k_{eff} — коэффициент размножения

Главное число всей реакторной физики. Сколько нейтронов в следующем поколении приходится на один нейтрон предыдущего.

k < 1 — цепочка затухает (подкритичен)

k = 1 — держится ровно (критичен, рабочий режим)

k > 1 — нарастает (надкритичен)

В программе — главный результат расчёта:

```
sp = openmc.StatePoint('statepoint.100.h5')
k = sp.keff                # например: 1.00241 +/- 0.00085
print(k.nominal_value, k.std_dev)
```

Реактивность

То же самое, но в удобной форме: насколько k отличается от единицы. Считается как (k − 1) / k. Нулевая реактивность — реактор критичен. Измеряется в долях или в специальных единицах (пкм, доллары).

Поток нейтронов (flux)

Сколько нейтронов пролетает через точку. Поле: в каждой точке зоны своё значение. Именно это поле ты будешь рисовать цветом в 3D. Там, где поток выше, — больше делений и сильнее нагрев.

Сечение (cross section)

Вероятность, что нейтрон провзаимодействует с ядром, выраженная как эффективная площадь мишени. Измеряется в барнах (1 барн = 10^{-24} см²). У каждого изотопа своё сечение, и оно сильно зависит от энергии нейтрона. Это главные входные данные расчёта — их берут из библиотек, а не вычисляют.

> Практический момент: библиотеки сечений ставятся отдельно от OpenMC и весят гигабайты. Без них расчёт не запустится — это первая вещь, на которой все спотыкаются при установке.

Быстрые и тепловые нейтроны

Только что родившийся нейтрон — быстрый (около 2 МэВ). После сотни столкновений с водой он тормозится до тепловой энергии (около 0,025 эВ) — то есть до скорости теплового движения молекул. Именно тепловые нейтроны и делят U-235.

Спектр
Распределение нейтронов по энергиям. Два базовых типа реакторов: тепловые (с замедлителем, работают на медленных нейтронах) и быстрые (без замедлителя). Физика у них разная.

Эффект Доплера
Топливо нагрелось → ядра сильнее колеблются → им проще поймать нейтрон → поглощение растёт → цепочка душится. Главная встроенная защита: реактор разгоняется, греется и глохнет сам, без всякой электроники.

> Отличный момент для демки: добавь слайдер температуры топлива — и покажи, как k_eff падает при нагреве. Это объясняет, почему реактор вообще безопасен.

Температурный коэффициент реактивности
Численная мера предыдущего эффекта: на сколько меняется реактивность при нагреве на один градус. Для безопасности обязан быть отрицательным.

Отравление ксеноном
Ксенон-135 — осколок деления с чудовищным сечением поглощения. Накапливается при работе, распадается при остановке. Создаёт медленные колебания с периодом в часы: после снижения мощности ксенон сначала растёт, потом спадает.

Выгорание (burnup)
Изменение состава топлива со временем работы: U-235 расходуется, копятся осколки, нарастает плутоний. Считается шагами: посчитали поле → пересчитали состав → снова поле.

> Для первой версии проекта выгорание не трогай. Свежее топливо, фиксированный состав — этого достаточно. OpenMC это умеет, но считает в разы дольше.

4. Расчёт: как это считается
Термины, которые встретятся в документации и в логах OpenMC.

Метод Монте-Карло
Напрямую уравнение переноса нейтронов не решить. Поэтому считают перебором: запускают виртуальный нейтрон и следят за его судьбой — пролетел, столкнулся, рассеялся, поглотился. Потом следующий. Миллионы раз. Статистика набирается — получается ответ.

История (history)
Жизнь одного виртуального нейтрона от рождения до гибели. Главное свойство: истории независимы друг от друга — поэтому задача идеально ложится на GPU, где тысячи потоков считают тысячи историй одновременно.

Циклы: неактивные и активные (inactive / active batches)
Расчёт идёт поколениями. Первые несколько десятков — неактивные: они нужны, чтобы распределение нейтронов «устаканилось», и в статистику не идут. Дальше активные — вот по ним и считается ответ.

В программе:
```
settings = openmc.Settings()
settings.particles = 10000     # нейтронов в одном поколении
settings.batches   = 150       # всего поколений
settings.inactive  = 50        # первые 50 выбрасываем
```

Сходимость источника
Та самая «устаканка». Если неактивных циклов мало, ответ будет смещён — и выглядеть при этом абсолютно правдоподобно. Классическая ошибка новичка.

Статистическая погрешность
Монте-Карло даёт не точное число, а число с разбросом. Погрешность падает как корень из числа историй: чтобы уменьшить её вдвое, нужно в четыре раза больше нейтронов. Результат без указанной погрешности — бессмысленный результат.

Tally (счётчик)
То, что ты просишь посчитать по ходу симуляции: поток, скорость делений, энерговыделение. Без tally расчёт выдаст только k_eff.

Поле потока на сетке — это и есть то, что ты будешь рисовать:

```
mesh = openmc.RegularMesh()
mesh.dimension = (50, 50, 20)
mesh.lower_left, mesh.upper_right = (-10, -10, 0), (10, 10, 40)

t = openmc.Tally(name='flux')
t.filters = [openmc.MeshFilter(mesh)]
t.scores  = ['flux', 'fission']
```

Библиотека сечений (ENDF / HDF5)
Файлы с измеренными данными по взаимодействию нейтронов с каждым изотопом. Международные, открытые, огромные. OpenMC их только читает.

Pin cell (топливная ячейка)
Простейшая модель: одна таблетка + оболочка + вода вокруг, с зеркальными границами (как будто она в бесконечной решётке таких же). С неё начинают все. Твой первый расчёт — именно она.

Бенчмарк
Эталонная задача с опубликованным ответом. Посчитал, сошлось — значит, модель и установка правильные. Не сошлось — ищешь ошибку у себя. Единственный честный способ проверить, что ты не считаешь чепуху.

5. Твоя часть: суррогат и визуализация
Термины из ML-половины проекта.

Суррогатная модель
Нейросеть, обученная повторять результат честного расчёта. Прогнал OpenMC несколько тысяч раз с разными параметрами — получил пары «вход → выход» — обучил на них модель. Дальше она отвечает мгновенно, ничего по-настоящему не считая. Точность чуть ниже, скорость выше в сотни и тысячи раз.

> Это и есть ядро твоего проекта: интерактив возможен только потому, что отвечает суррогат, а не солвер.

Входы и выходы (для твоей модели)
Вход: обогащение, положение управляющих стержней, температура топлива и воды, шаг решётки, геометрия.
Выход: k_eff (одно число) и поле потока (трёхмерный массив).

Пространство параметров и выборка

Набор комбинаций входов, на которых ты гоняешь честный расчёт. Брать равномерную сетку по всем параметрам неэффективно: при пяти параметрах по десять точек — сто тысяч прогонов. Берут латинский гиперкуб (Latin Hypercube) — он покрывает пространство равномерно, но гораздо меньшим числом точек.

Обучающая и тестовая выборки

Делишь данные: на одной части обучаешь, на другой проверяешь. Модель не должна видеть тестовые точки при обучении — иначе метрики будут красивыми и ложными.

Метрика

Чем измеряешь качество. Для k_eff — абсолютная ошибка (важна третья цифра после запятой). Для поля потока — относительная ошибка по всем точкам сетки. Опубликованные работы дают ориентир: ошибка поля порядка 3–4% считается хорошим результатом.

Цифровой двойник (digital twin)

Общее название для того, что ты делаешь: модель установки, работающая достаточно быстро, чтобы отвечать в реальном времени на изменение параметров.

Порядок действий

Что делать первым, чтобы термины перестали быть словами.

1	Поставить OpenMC и библиотеку сечений. Заложи пару дней на боль.
2	Посчитать pin cell. Получить k_eff с погрешностью.
3	Сверить с опубликованным бенчмарком. Сошлось — всё поставлено верно.
4	Добавить tally на поток, нарисовать срез. Первая картинка.
5	Автоматизировать прогон: скрипт меняет параметры, складывает результаты в базу.
6	Латинский гиперкуб → тысячи прогонов → датасет.
7	Обучить базовый суррогат. Проверить на отложенной выборке.
8	3D в браузере со слайдерами. Вот теперь это демка.

> Самое долгое здесь — не код, а понимание, какие параметры вообще осмысленно крутить и в каких пределах. Это нормально. Шаг 3 — ключевой: пока бенчмарк не сошёлся, всё остальное считать рано.